

The Language Contract & The Memory Wall: ISA, RISC/CISC, and Caches

If the CPU is the thinking machine, the ISA is the language it thinks in. And if the CPU is the sprinter, memory is the starting blocks it's chained to. This lecture covers both.

What Is an ISA? The Hardware–Software Contract

There's an Android phone made by Samsung (Exynos chip) and one made by Google (Tensor chip). Different silicon. Different transistors. Different microarchitectures. Yet the same Android app runs on both. How? The answer: they both implement the same ISA — ARM64 / AArch64. The ISA is the contract.

Instruction Set Architecture (ISA)

The ISA is the complete specification of the interface between a processor and the software that runs on it. It defines what operations a processor can perform, how those operations are encoded as bit patterns, how memory is organized and addressed, and what programmer-visible state (registers, flags) exists. The ISA is the contract: hardware engineers implement it; software engineers program to it.

Crucially, the ISA says nothing about *how* the hardware implements the operations. Two processors can implement the same ISA with completely different microarchitectures — different pipeline depths, different cache sizes, different execution units — yet run the same binary. This is why x86 software compiled in 1994 still runs on a 2025 Intel processor, even though the internal hardware is completely different.

The ISA as a contract is also why changing an ISA is so expensive. The x86 ISA has been extended (but never truly broken) since 1978. Billions of programs depend on it. The cost of changing a deployed ISA is measured in the software ecosystem, not the hardware.

What an ISA Specifies: The Six Components

1. The Instruction Set (Opcodes & Operations)

The list of all operations the processor can perform, each identified by an **opcode** (a numeric code). Operations include arithmetic (ADD, SUB, MUL, DIV), logical (AND, OR, XOR, NOT, SHL, SHR), data movement (LOAD, STORE, MOV), control flow (JMP, BEQ, BNE, CALL, RET), and system operations (interrupt, privilege change).

2. Instruction Formats

How each instruction is encoded as a bit pattern. The format specifies the width of the instruction (32-bit in MIPS, variable-length in x86) and the fields within it (opcode field,

register fields, immediate fields, offset fields). The format design is a core engineering tradeoff: wider instructions allow more expressive operations, but cost code density. Narrower instructions pack more tightly, but limit what you can express in a single operation.

3. The Register Set

How many programmer-visible registers exist, how wide they are, and whether any have special roles. MIPS32 has 32 general-purpose 32-bit registers (\$0–\$31). x86-64 has 16 general-purpose 64-bit registers. ARM64 has 31 general-purpose 64-bit registers. More registers allow the compiler to keep more values "in flight" without touching memory, reducing load/store instructions.

Some registers have architectural special roles: the stack pointer (SP), link register (LR, for function return addresses), the zero register (MIPS \$0 / RISC-V x0, always reads as 0). These conventions are part of the ISA specification.

4. Data Types & Sizes

What data types the ISA natively supports: 8-bit bytes, 16-bit halfwords, 32-bit words, 64-bit doublewords, single-precision floats (32-bit), double-precision floats (64-bit). Each may have separate load/store instructions and alignment requirements.

5. Addressing Modes

How the ISA specifies memory addresses in instructions. Common modes:

- **Register direct:** operand is in a register. `ADD R1, R2, R3`
- **Immediate:** constant value embedded in the instruction. `ADDI R1, R2, 5` (add 5)
- **Base + offset:** address = register + constant. `LW R1, 8(R2)` (load from R2+8). The workhorse of RISC memory access.
- **PC-relative:** address = PC + offset. Used for branches and jumps. `BEQ R1, R2, +12` (if R1==R2, jump 12 bytes forward).
- **Indirect (CISC):** the register holds the address of the address. `MOV AX, [BX+SI*4+8]` (x86). Powerful but hardware-expensive to decode.

6. Memory Model

How memory is organized: byte-addressable or word-addressable? **Little-endian** (least significant byte at lowest address — Intel x86, ARM) or **big-endian** (most significant byte first — old MIPS, network protocols)? Are unaligned memory accesses allowed? What are the cache coherence guarantees? These details matter enormously in systems programming and multi-core correctness.

3.3

MIPS32: A Complete ISA Case Study

MIPS32 is the gold standard for teaching ISAs — clean, well-specified, historically important (used in early PlayStation, Silicon Graphics workstations), and the basis on which RISC-V was designed. We'll use it to make all the abstract ISA concepts concrete.

MIPS32 has **32 registers** (\$0–\$31), **fixed 32-bit instruction width**, and three instruction formats.

The Three MIPS Instruction Formats

Look up about R-type, I-type and J-type instruction formats.

Key design insight from MIPS formats: All registers are 5 bits wide (selecting 1 of 32 registers). The I-type immediate is 16 bits, meaning constants in the range 32768 to +32767 can be encoded directly. The J-type uses 26 bits for the target, giving a 256MB jump range (26 bits $\times$ 4 bytes). For 32-bit full-range jumps, MIPS provides a register-indirect jump: JR \$ra.

Key MIPS Instructions with Encoding

Mnemonic	Format	Operation	Example
add rd,rs,rt	R	rd = rs + rt (signed)	add \$t0,\$t1,\$t2
addi rt,rs,imm	I	rt = rs + sign_ext(imm)	addi \$t0,\$t1,10
lw rt,imm(rs)	I	rt = Mem[rs + imm]	lw \$t0,4(\$sp)
sw rt,imm(rs)	I	Mem[rs + imm] = rt	sw \$t0,4(\$sp)
beq rs,rt,offset	I	if rs==rt: PC += offset $\times$ 4+4	beq \$t0,\$t1,loop
j target	J	PC = PC[31:28] : target : 00	j exit
slt rd,rs,rt	R	rd = (rs < rt) ? 1 : 0	slt \$t0,\$t1,\$t2
lui rt,imm	I	rt = imm $\ll$ 16	lui \$t0,0xDEAD

The \$zero Register Trick

MIPS register \$0 (also written \$zero) is hardwired to always read as 0 and silently discards any write. This isn't wasted — it gives the ISA several "free" pseudo-instructions:

- move \$t0, \$t1 $\rightarrow$ add \$t0, \$zero, \$t1 (copy register)
- beqz \$t0, label $\rightarrow$ beq \$t0, \$zero, label (branch if zero)
- li \$t0, 5 $\rightarrow$ addi \$t0, \$zero, 5 (load small immediate)
- nop $\rightarrow$ sll \$zero, \$zero, 0 (no operation)

By dedicating one register to zero, the instruction set becomes smaller and cleaner. RISC-V takes the same approach with x0. *This is a key insight for the assignment.*

RISC vs. CISC: A Philosophy War

By the late 1970s, the question of how to design an ISA had split into two distinct philosophies. The debate was resolved by the market, but understanding both sides explains every ISA decision made in the past 40 years.

CISC — Complex Instruction Set Computing

The CISC philosophy (Intel x86, VAX) argues that more powerful instructions reduce program size, which matters when memory is expensive. A CISC processor might have a single instruction that: loads two values from memory, multiplies them, adds a constant, and stores the result back. One instruction, multiple operations. x86's LOOP instruction decrements a counter AND branches in one opcode.

CISC instructions have variable lengths (x86 instructions are 1–15 bytes). The decoder must figure out where each instruction ends before it can begin the next fetch. This makes the decoder complex, power-hungry, and hard to pipeline at high speed.

RISC — Reduced Instruction Set Computing

The RISC philosophy (MIPS, ARM, RISC-V), formalized by Patterson and Hennessy at UC Berkeley and Stanford in the 1980s, argues that simplicity enables speed. The guiding principles:

- **Fixed instruction length.** Every instruction is the same width. The pipeline stage that fetches and aligns instructions is trivially simple and fast.
- **Load-store architecture.** Only LOAD and STORE instructions access memory. ALL other instructions (ADD, AND, branch) operate only on registers. This simplifies the execute stage — the ALU always operates on registers, never on memory operands.
- **Single-cycle execution (mostly).** Most instructions complete in one pipeline stage. The exception is loads (multi-cycle memory access), which is why the load-use hazard exists.
- **Large, uniform register set.** More registers → fewer memory accesses → faster code.
- **Simple addressing modes.** Only base+offset for memory; no indirect, scaled, or segment-based modes. Simplicity allows faster address computation.
- **Hardwired, not microcoded.** CISC processors often use a microcode ROM to translate complex instructions into internal micro-operations. RISC processors execute directly in hardware, faster and with lower power.

- **Compiler-friendly.** RISC relies on the compiler to optimize instruction sequences rather than providing hardware shortcuts. The bet: compilers will get better.

The Modern Convergence

The RISC/CISC debate is now largely academic, because x86 — the dominant desktop and server ISA — is CISC on the outside and RISC on the inside. Intel's Pentium Pro (1995) introduced a "decode" front-end that translates x86 instructions into simpler internal **micro-operations** (**μops**) that look very much like RISC instructions. These μops then flow through a RISC-style pipelined backend.

ARM (the dominant mobile ISA) is RISC. RISC-V (the rising open-source ISA, used in NVIDIA GPUs, Western Digital drives, and many SoCs) is pure RISC. The trend in new ISAs is firmly RISC. The win of x86 is a historical accident (IBM PC standardization), not a technical one.

The Memory Wall: Why Fast CPUs Still Starve

A modern CPU core can execute ~4 billion operations per second. DRAM (your RAM) can supply data at a peak of maybe 50 billion bytes per second — but with a latency of 60–80 nanoseconds per access. In those 70ns, the CPU could have executed ~280 instructions. Instead it waited. This is the memory wall.

In 1980, processor speed and memory speed were within a factor of 2 of each other. Processor performance then doubled every 18 months (Moore's Law + Dennard scaling). DRAM speed improved only about 7% per year. By 2000, the gap had grown to 100×. Today it's over 1000×.

The implication: adding more execution units or a faster pipeline to the CPU makes almost no difference to real application performance if the CPU is spending most of its time waiting for data from memory. The bottleneck is off-chip, and no amount of on-chip optimization can fix it.

Memory Latency – Roughly

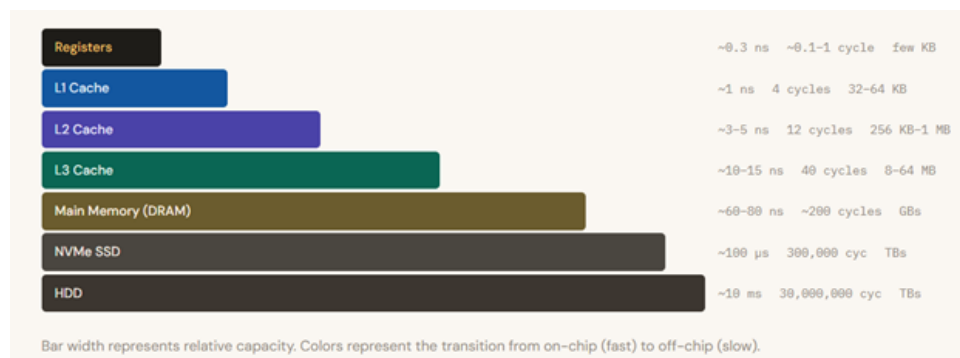


Figure 1:

SRAM vs. DRAM: Why Two Technologies?

SRAM (Static RAM) stores each bit using 6 transistors in a cross-coupled latch. The state is held by the circuitry itself — no refresh needed. Reads are fast (no capacitor charging/discharging) and non-destructive. But 6 transistors per bit is expensive and area-hungry. SRAM is used for registers and caches — where you need maximum speed.

DRAM (Dynamic RAM) stores each bit as a charge on a capacitor, controlled by a single transistor — hence "1T1C." This is $6\times$ denser than SRAM, making large main memory affordable. But there are two penalties: (1) the capacitor leaks, requiring a periodic **refresh** operation that temporarily blocks access; (2) reading a DRAM cell is destructive — the sense amplifier drains the capacitor and must then rewrite the bit. This, combined with the row activation latency of DRAM banks, creates the multi-cycle DRAM access latency.

Caches: Exploiting Locality to Bridge the Gap

Cache

A small, fast SRAM memory that sits between the CPU and main DRAM, storing copies of recently or frequently accessed data. When the CPU needs data, it first checks the cache. A **cache hit** means the data is there — served in a few cycles. A **cache miss** means the data must be fetched from the next level of memory — costly, but the data is then placed in the cache for next time.

Caches work because real programs exhibit two powerful patterns:

- **Temporal locality:** if you accessed address X recently, you'll likely access X again soon. (Example: a loop variable accessed every iteration.)
- **Spatial locality:** if you accessed address X, you'll likely access addresses near X soon. (Example: iterating through an array — accessing element [i] makes [i+1] very likely next.)

Caches exploit both: a cache miss doesn't just bring the requested word — it fetches an entire **cache line** (typically 64 bytes, or 16 words). This exploits spatial locality. Temporal locality is exploited by keeping recently used lines in cache and only evicting them when space is needed.

How a Direct-Mapped Cache Works

The simplest cache structure: each memory address maps to exactly one cache slot. A memory address is split into three fields:

- **Block offset:** selects which byte within the 64-byte cache line.
- **Index:** selects which cache slot (or "set") to look in. If the cache has 256 sets, the index is 8 bits.

- **Tag:** the remaining high-order bits, stored alongside the data to verify which memory address is currently occupying this cache slot.

On a memory access: the hardware extracts the index, looks up that cache slot, compares the stored tag to the address's tag bits, and checks the **valid bit**. If they match: cache hit. If not: cache miss, fetch from next level, update slot.

The direct-mapped problem: conflict misses

If two frequently-used arrays both map to the same cache set (because their addresses differ by exactly the cache size), they'll repeatedly evict each other — a **thrashing** pattern. Every access to array A evicts array B from that set, and vice versa. The cache can be half-empty, yet still miss on every access. This is why set-associativity (covered in Lecture 4) is critical.